Assignment 2 Q3: Results and analysis

Assignment 2 Q3: Algorithm comparison and design

Experiment

We compared CUBIC, Reno, and BBR at three destinations, with three runs per algorithm and destination. Every run lasted 60 seconds and sampled the data socket once per second. This gives 27 matched runs and 1,620 samples. The run identifier is q3-test. Despite the name, these are full 60-second experiments, not short smoke tests. Each algorithm was selected before connecting the data socket, then read back from that socket and saved with the measurements.

The host ran Linux 6.18.33.2-microsoft-standard-WSL2, Python 3.14.8, matplotlib 3.11.2, and pydantic 2.13.5. The manifest confirms that the kernel accepted cubic, reno, and bbr. It does not identify the BBR implementation version or whether the run used Docker, so we do not assume either. The supplied Dockerfile uses ubuntu:24.04; reproducing the kernel behavior also requires a supported Linux/WSL2 host with the algorithms available.

We shuffled the server list with seed 422 and rotated the initial algorithm order across repetitions. Dallas used BBR/CUBIC/Reno, CUBIC/Reno/BBR, then Reno/BBR/CUBIC. Netfiber used Reno/BBR/CUBIC, BBR/CUBIC/Reno, then CUBIC/Reno/BBR. The IP destination used CUBIC/Reno/BBR, Reno/BBR/CUBIC, then BBR/CUBIC/Reno. Thus each algorithm occupied each position once at each retained destination. Individual tests were sequential, with a configured two-second pause between them. The representative destination is dal.speedtest.clouvider.net:5200, the first complete group; it was chosen without comparing throughput.

Eight candidate destinations were attempted. Of 72 tests, 49 succeeded and 23 failed. The failures included server rejections, timeouts, broken pipes, and an unreachable network. Five destinations lacked at least one successful repetition. Their 22 successful runs are retained as partial data, but are excluded from the matched comparisons below. All three requested destination groups completed. The exact execution order, timestamps, algorithms, and failures are in manifest.json and experiment.log.

Assignment 2 Q3: Results and analysis

Measurement and summary method

Goodput is the increase in the data socket's cumulative acknowledged-byte count divided by the actual sampling interval, multiplied by eight. We report Mbit/s. A run's duration-weighted goodput is total acknowledged bytes divided by total sampled seconds. Each reported algorithm mean gives equal weight to its three run means. The variability shown with those means is the sample standard deviation across the three runs, not a confidence interval.

RTT is Linux TCP_INFO's smoothed RTT in milliseconds. A run's mean RTT averages its 60 samples; the algorithm summary averages the three run means. Congestion window is snd_cwnd in segments. Its byte budget is snd_cwnd times snd_mss; it is a limit on outstanding data, not a direct measurement of bytes currently in flight. Retransmissions are interval increases in total_retrans and are a loss proxy, not an independent measurement of packet-loss probability.

Per-run summary tables include minimum, median, arithmetic mean, and P95 sampled goodput for every destination and algorithm. P95 is linearly interpolated at index (n - 1) times 0.95 in the sorted samples. The CSV also includes duration-weighted goodput, mean/P95 RTT, window statistics, and an observed-rate bandwidth-delay-product proxy. Startup and low-throughput intervals remain in every summary; no outliers were removed.

Dallas: throughput and RTT tradeoff

At dal.speedtest.clouvider.net:5200, mean goodput was 281.30 Mbit/s for Reno, 273.96 for CUBIC, and 269.79 for BBR. The corresponding across-run standard deviations were 2.76, 5.63, and 11.69 Mbit/s. Reno's mean was 2.7% above CUBIC and 4.3% above BBR. BBR had the lowest mean RTT: 64.53 ms, versus CUBIC's 68.73 and Reno's 73.65 ms. That is a throughput/latency tradeoff in these measurements, not a universal ranking. With only three runs and overlapping throughput ranges, the ordering needs more evidence before being treated as a consistent advantage.

Assignment 2 Q3: Results and analysis

The representative plots show different window trajectories without proportional changes in delivered rate. Reno's median cwnd increased across repetitions from 1,982 to 4,657 to 7,854 segments, while its run goodputs stayed between 279.14 and 284.41 Mbit/s. In its third run, median cwnd times MSS was about 11.37 MB, while achieved rate times minimum filtered RTT was about 1.09 MB. A larger window budget therefore did not translate into proportionally higher goodput. Another limit, such as the receiver, server, or path capacity, may dominate; the trace does not identify which one.

Minimum filtered RTTs were around 29-32 ms, while sampled mean RTTs were roughly 59-75 ms across runs. The excess is consistent with queueing or other variable delay, including reverse-path delay. We cannot assign all of it to a forward-path queue. Across the three runs, Reno recorded 444 retransmitted segments, CUBIC 224, and BBR 196. Reno still delivered the highest mean goodput, so a larger retransmission count alone does not determine the throughput ordering.

Netfiber: long RTT and a variable BBR run

At speed.netfiber.net.il:5201, minimum filtered RTTs were around 170-174 ms. Reno and CUBIC achieved similar mean rates: 119.07 and 118.08 Mbit/s, with standard deviations of 0.89 and 4.16 Mbit/s. Reno's advantage was only 0.8%, smaller than the observed CUBIC run variation. Their mean RTTs were also similar, at 181.67 and 181.90 ms.

BBR averaged 106.59 Mbit/s with a much larger 17.37 Mbit/s standard deviation, while its mean RTT was slightly lower at 177.39 ms. Its first two runs delivered 115.56 and 117.63 Mbit/s with two and zero retransmitted segments. The third delivered 86.57 Mbit/s with 396 retransmissions, accounting for almost all of BBR's 398 retransmissions at this destination. At about 17 seconds in that run, goodput was 33.16 Mbit/s with RTT 176.11 ms, close to the path's baseline. This associates the low-goodput period with retransmission activity and window changes without evidence of sustained RTT inflation. It does not prove whether random loss, cross-traffic, or another condition caused it.

The median window sizes across all samples were 5,820 segments for Reno and 3,964 for CUBIC, despite their similar rates. As in Dallas, extra permitted in-flight data was not sufficient to raise useful throughput. This path is a high-RTT regime where Reno and CUBIC performed similarly in our runs; it provides no measured evidence that CUBIC's large-bandwidth-delay scaling produced an advantage here.

Assignment 2 Q3: Results and analysis

IP destination: similar Reno/CUBIC rates and transient stalls

At 96.45.41.167:30001, Reno and CUBIC averaged 163.52 and 163.12 Mbit/s. Their standard deviations were 11.80 and 17.51 Mbit/s, making the 0.41 Mbit/s difference negligible relative to run variability. CUBIC had a slightly lower mean RTT, 132.58 ms versus Reno's 136.69 ms, but the respective RTT standard deviations were both about 13.3 ms.

BBR averaged 147.47 Mbit/s and 150.60 ms RTT. Its RTT standard deviation was 38.60 ms, driven partly by a 192.79 ms mean in its first run. At about 50 seconds in that run, acknowledged goodput was zero for one interval and smoothed RTT reached 1,559.64 ms; retransmissions also increased. This is a short stall within a completed transfer, not a failed experiment. BBR's three runs recorded 541 retransmitted segments in total, versus CUBIC's 120 and Reno's 11.

The median window across samples was 5,749 segments for BBR, versus 3,965 for CUBIC and 3,984 for Reno. Here the larger BBR window coexisted with lower mean goodput and more variable RTT. That combination supports using throughput and loss signals alongside window size. It does not establish that BBR caused the path conditions: the algorithms were tested at different times, and their minimum RTTs also differed.

Operating regimes and limits of the comparison

Reno increases its congestion window by approximately one segment per RTT during congestion avoidance [1]. CUBIC uses a cubic growth function intended to scale on fast, long-distance paths [2]. BBR uses a delivery-rate and RTT model to guide sending [3]. Those mechanisms explain what each algorithm tries to do; they do not override the measured results or identify the cause of a public-path anomaly.

In this dataset, Reno had the highest mean goodput at each destination. Its advantage over CUBIC was small on Netfiber and negligible on the IP destination. BBR had the lowest mean RTT on Dallas and Netfiber, with lower throughput there, and did not maintain that RTT advantage on the IP destination. No sampled destination establishes a CUBIC throughput win or a BBR throughput win. CUBIC's expected advantage on a controlled large-bandwidth-delay bottleneck, or BBR's possible advantage under particular loss/queue conditions, would require additional controlled tests rather than being inferred from these traces.

Assignment 2 Q3: Results and analysis

For a window-limited sender, rate is approximately cwnd times MSS divided by RTT. Bandwidth-delay product is bottleneck bandwidth times propagation RTT. Our recorded proxy substitutes achieved goodput and minimum filtered RTT, so it is not an independent estimate of bottleneck capacity. On the third Dallas Reno run, the window budget was about 10.4 times that proxy; this demonstrates that window budget alone cannot explain the rate, but does not measure queue occupancy. Linux's receive-window, pacing, recovery, and application limits can impose tighter constraints than cwnd.

The scatter plots pair instantaneous window/RTT readings with goodput and retransmissions over the preceding interval. Recovery can affect later intervals, so scatter associations are not causal estimates. The one-second sampling can also miss shorter congestion events. Differences in Internet cross-traffic, routes, reverse-path traffic, server limits, and host load remain uncontrolled. Rotated order reduces position bias but cannot make sequential tests simultaneous. Selecting only complete server groups favors reliable destinations. Three repetitions provide limited evidence about variability, and these were single-flow tests, not fairness experiments.

Algorithm design

Q3_DESIGN.md supplies the state, pseudocode, initial parameters, and fairness/stability discussion for our proposed algorithm. The attached design uses RTT, acknowledged throughput, and recovery/loss events together. The baseline results motivate its decision to stop routine window growth when a larger budget is not improving delivery, to distinguish sustained RTT inflation from loss activity, and to keep occasional probes when an observed rate estimate may be too low. Its thresholds remain hypotheses; none of the measurements evaluate the proposed algorithm itself. The next assignment must implement and test it.

Reproduction and implementation links

From the Assignment 2 directory, install requirements.txt and run: python main.py compare -s iperf3-servers.csv -c 3 -t 60 -g 1 -r 3 --seed 422 --run-id q3-test -o results. Use a fresh run ID if that directory already exists. This reproduces the experiment procedure; changing public Internet conditions will change results. The recorded Python and library versions above describe the submitted run; requirements.txt pins a different locally tested environment, so it should not be presented as the exact measured user-space environment.

Assignment 2 Q3: Results and analysis

The input snapshot is results/q3-test/servers-input.csv. Raw per-test CSV/JSON, actual algorithms, run identifiers, units, kernel version, failure diagnostics, per-run summaries, and aggregate summaries are in that directory. validation.json records an independent consistency check of the imported archive. Rebuild the plots and reports offline with: python main.py analyze --manifest results/q3-test/manifest.json. The reviewed Q3 report is q3-report.pdf. Q3_RUN.md documents the Docker command and remaining team submission steps.

Repository: https://github.com/aunali0/CS422_assignment_2 . Socket selection before connect: https://github.com/aunali0/CS422_assignment_2/blob/4389022/client.py#L53 . Experiment ordering and persistence: https://github.com/aunali0/CS422_assignment_2/blob/4389022/comparison.py#L51 . Measurement and common-scale plots: https://github.com/aunali0/CS422_assignment_2/blob/4389022/comparison_plots.py#L22 and https://github.com/aunali0/CS422_assignment_2/blob/4389022/comparison_plots.py#L74 . These links identify the experiment implementation; reviewed_report.py generates the final Q3 PDF from these observations, the saved results, and the design.

References and assistance

[1] RFC 5681, TCP Congestion Control, section 3.1: https://www.rfc-editor.org/rfc/rfc5681.html#section-3.1

[2] RFC 9438, CUBIC for Fast and Long-Distance Networks: https://www.rfc-editor.org/rfc/rfc9438.html

[3] Google's BBR FAQ, model overview: https://github.com/google/bbr/blob/master/Documentation/bbr-faq.md . The saved kernel algorithm name is bbr; its implementation version was not recorded.

AI assistance was used for experiment automation, plotting, consistency checks, and drafting the design and analysis. The measurements were collected on a team member's Linux/WSL2 machine. The proposed algorithm has not yet been evaluated.

Matched runs: means and sample standard deviations

Destination	Algorithm	Goodput (Mbit/s)	RTT (ms)	Retransmissions
D1	cubic	273.96 +/- 5.63	68.73 +/- 5.77	224
D1	reno	281.30 +/- 2.76	73.65 +/- 0.41	444
D1	bbr	269.79 +/- 11.69	64.53 +/- 6.62	196
D2	cubic	118.08 +/- 4.16	181.90 +/- 5.06	23
D2	reno	119.07 +/- 0.89	181.67 +/- 2.96	41
D2	bbr	106.59 +/- 17.37	177.39 +/- 1.13	398
D3	cubic	163.12 +/- 17.51	132.58 +/- 13.37	120
D3	reno	163.52 +/- 11.80	136.69 +/- 13.29	11
D3	bbr	147.47 +/- 6.02	150.60 +/- 38.60	541

Per-run sampled goodput (Mbit/s)

Destination	Algorithm	Run	Min	Median	Mean	P95
D1	bbr	1	202.81	260.73	256.32	281.47
D1	cubic	1	208.03	275.57	273.69	287.34
D1	reno	1	219.24	281.23	279.15	293.89
D1	cubic	2	222.77	281.35	279.73	292.68
D1	reno	2	220.85	285.47	284.41	296.55
D1	bbr	2	197.25	282.65	277.17	295.82
D1	reno	3	217.91	283.04	280.35	297.14
D1	bbr	3	209.80	281.98	275.89	297.12
D1	cubic	3	17.66	277.09	268.47	292.97
D2	reno	1	3.59	122.91	118.04	127.96
D2	bbr	1	2.51	120.57	115.56	128.97
D2	cubic	1	3.59	122.35	120.06	127.93
D2	bbr	2	2.46	123.70	117.63	129.11
D2	cubic	2	3.59	117.32	113.31	123.80
D2	reno	2	3.59	122.63	119.49	126.31

Per-run sampled goodput (Mbit/s)

Destination	Algorithm	Run	Min	Median	Mean	P95
D2	cubic	3	3.59	123.45	120.89	128.41
D2	reno	3	3.59	122.84	119.67	127.14
D2	bbr	3	2.18	90.94	86.57	120.68
D3	cubic	1	23.36	174.92	169.73	179.38
D3	reno	1	24.36	180.37	176.22	185.59
D3	bbr	1	0.00	176.74	148.14	186.68
D3	reno	2	16.49	159.57	152.89	183.65
D3	bbr	2	12.03	164.48	153.13	180.58
D3	cubic	2	12.57	156.31	143.27	170.21
D3	bbr	3	2.24	183.42	141.15	190.99
D3	cubic	3	29.35	186.27	176.35	193.78
D3	reno	3	20.13	178.49	161.47	186.67

Destination key and implementation links

D1: dal.speedtest.clouvider.net:5200

D2: speed.netfiber.net.il:5201

D3: 96.45.41.167:30001

Repository

Data socket algorithm selection: client.py

Experiment runner: comparison.py

Statistics: comparison_plots.py

Common-scale figures: comparison_plots.py

reviewed_report.py assembles this PDF from observations.md, raw results, and Q3_DESIGN.md.

The linked commit identifies the original experiment code. Reviewed analysis is included with this report.


D1: dal.speedtest.clouvider.net:5200
Goodput (Mbit/s)
0
50
100
150
200
250
300
bbr r1
cubic r1
reno r1
cubic r2
reno r2
bbr r2
reno r3
bbr r3
cubic r3
D2: speed.netfiber.net.il:5201
Goodput (Mbit/s)
0
20
40
60
80
100
120
reno r1
bbr r1
cubic r1
bbr r2
cubic r2
reno r2
cubic r3
reno r3
bbr r3
D3: 96.45.41.167:30001
Goodput (Mbit/s)
0
25
50
75
100
125
150
175
200
cubic r1
reno r1
bbr r1
reno r2
bbr r2
cubic r2
bbr r3
cubic r3
reno r3
0
10
20
30
40
50
60
Time (s)

dal.speedtest.clouvider.net:5200 | matched settings; individual repetitions

cubic | reno | bbr

Congestion window (segments)

RTT (ms)

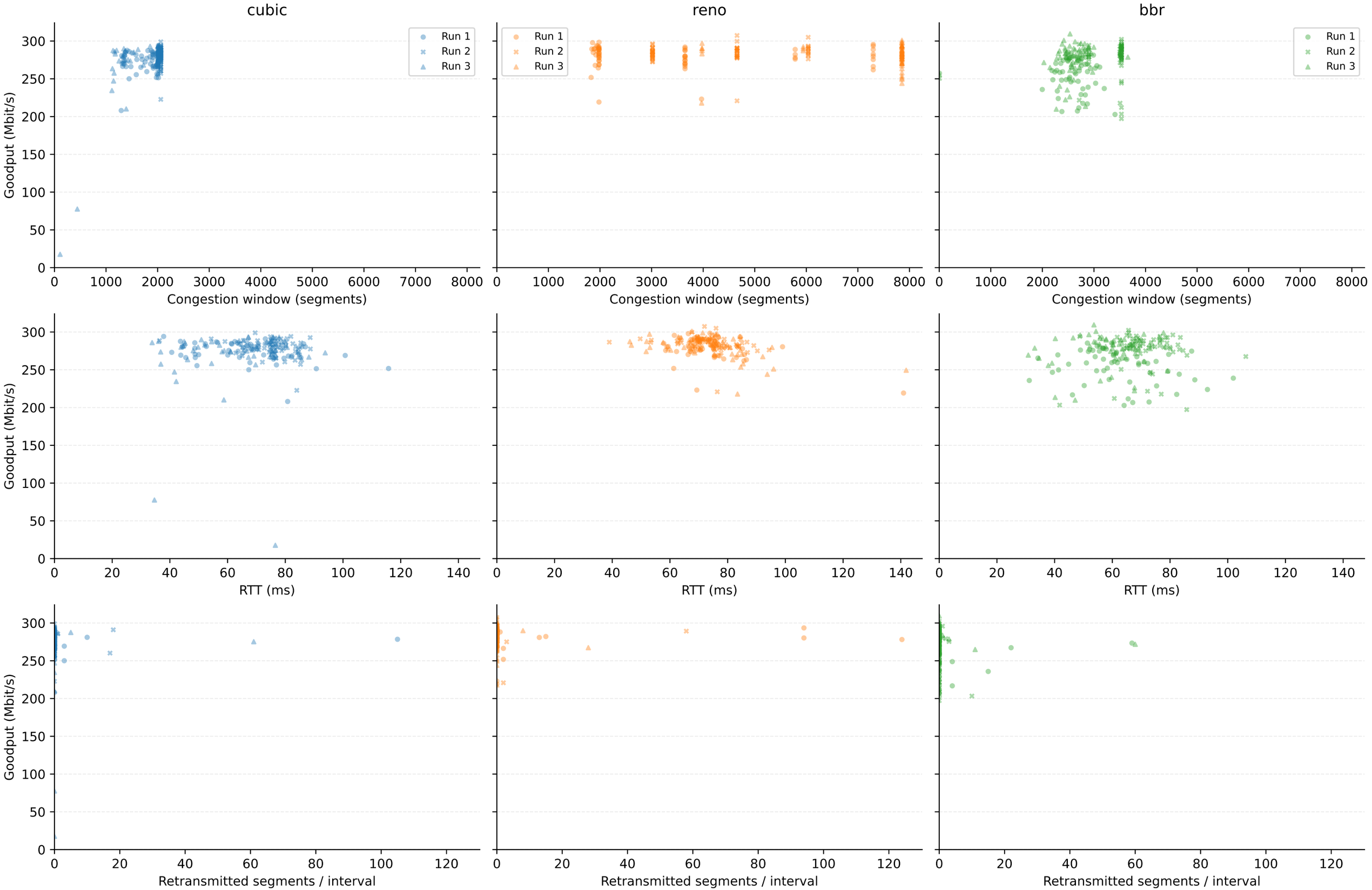
Retransmitted segments / interval

Goodput (Mbit/s)

Time (s)

Run 1
Run 2
Run 3

dal.speedtest.clouvider.net:5200 | all repetitions; common scales per metric



speed.netfiber.net.il:5201 | matched settings; individual repetitions

cubic

reno

bbr

Congestion window (segments)

RTT (ms)

Retransmitted segments / interval

Goodput (Mbit/s)

Time (s)

Run 1

Run 2

Run 3

speed.netfiber.net.il:5201 | all repetitions; common scales per metric

cubic

reno

bbr

Run 1
Run 2
Run 3

Goodput (Mbit/s)

Congestion window (segments)

RTT (ms)

Retransmitted segments / interval

96.45.41.167:30001 | matched settings; individual repetitions

cubic | reno | bbr

Congestion window (segments)

RTT (ms)

Retransmitted segments / interval

Goodput (Mbit/s)

Time (s)

Run 1
Run 2
Run 3

96.45.41.167:30001 | all repetitions; common scales per metric

cubic

reno

bbr

Run 1
Run 2
Run 3

Goodput (Mbit/s)

Congestion window (segments)

RTT (ms)

Retransmitted segments / interval

Assignment 2 Q3(b): Algorithm design

Q3(b): RTT, throughput, and loss based window control

Status: proposed design. The completed q3-test dataset compares CUBIC, Reno, and BBR at three destinations with three 60-second repetitions each. Those measurements informed the design rationale below; the proposed algorithm itself has not been implemented or measured. Assignment 3 must implement and evaluate it separately.

State and measurements

Maintain cwnd and ssthresh in segments, initialized to 10 and infinity; obey any lower kernel or peer limit. Minimum cwnd is 2. Let MSS be the current sending maximum segment size in bytes. Use the host TCP stack for ACK processing, retransmission, recovery, and timeout handling.

Update once per measured RTT, using only newly acknowledged data. During each epoch collect acknowledged bytes, elapsed time, RTT samples, and newly retransmitted segments. Goodput is acknowledged bytes / elapsed seconds, in bytes/s. Estimate throughput with an EWMA: rate = 0.75 * previous_rate + 0.25 * epoch_goodput; initialize it from the first valid epoch.

Let base_rtt be the minimum valid RTT in a rolling 10-second window. Let queue_ratio = max(0, srtt / base_rtt - 1). Keep a rolling maximum of non-application-limited epoch goodput over 10 seconds as bandwidth_estimate. Then bdp_segments = max(2, bandwidth_estimate * base_rtt / MSS). RTT values in this calculation are seconds. This is an observed delivery-rate estimate, not a measurement of physical link capacity.

A loss event means a newly entered TCP recovery episode, not every ACK that reports retransmissions. Only reduce once in that episode, respecting the kernel recovery boundary. Ignore epochs with no newly acknowledged bytes for growth; timeout handling still applies. Freeze delay and throughput growth decisions when the sender is application limited or receiver-window limited.

Assignment 2 Q3(b): Algorithm design

Pseudocode

```
State: cwnd = 10; ssthresh = infinity; mode = STARTUP; rate = unset
On valid ACK: accumulate delivered bytes and RTT samples for this epoch
On entering a new loss recovery episode:
    cwnd = max(2, floor(0.70 * cwnd))
    ssthresh = cwnd; mode = AVOIDANCE
    mark recovery boundary; reset epoch accounting
On retransmission timeout:
    ssthresh = max(2, floor(cwnd / 2))
    cwnd = 2; mode = AVOIDANCE; reset epoch accounting
Once per RTT, after ACK progress, outside loss recovery:
    update base_rtt, bandwidth_estimate, and rate
    if application limited or receiver-window limited: hold cwnd; finish epoch
    if mode == STARTUP:
        if queue_ratio < 0.10:
            cwnd = cwnd + min(newly_acked_segments, cwnd)
            finish epoch
        mode = AVOIDANCE; ssthresh = cwnd
    if queue_ratio > 0.25:
        cwnd = max(2, floor(0.90 * cwnd))
    else if queue_ratio < 0.10:
        if cwnd < 1.25 * bdp_segments or rate > 1.02 * previous_rate:
            cwnd = cwnd + 1
        else if at least 8 RTT epochs have passed since the last probe:
            cwnd = cwnd + 1
            reset probe timer
    else:
        hold cwnd
    enforce kernel cwnd clamp; finish epoch
```

Assignment 2 Q3(b): Algorithm design

Initialize previous_rate from the first epoch and skip the rate-growth comparison for that epoch. Count newly acknowledged segments with fractional credit across ACKs, so delayed ACKs and partial segments do not inflate growth. Maintain the probe timer across epochs, incrementing it once per eligible RTT and resetting it on a probe or reduction. Do not stack a delay reduction on a loss reduction within the same recovery epoch. A kernel/eBPF implementation must let the stack apply timeout behavior exactly once rather than performing a second reduction in an ACK callback.

Design choices

The existing CUBIC traces motivated retaining acknowledged-byte goodput, smoothed RTT, and retransmission counters as the observable inputs. Two 15-second runs to 185.180.12.40:5201 had duration-weighted goodputs of 164.20 and 93.62 Mbit/s, mean smoothed RTTs of 128.18 and 129.08 ms, and retransmission deltas totaling 0 and 147 segments. Their median congestion windows were 4165 and 1135.5 segments. These runs used the same algorithm: they show substantial variability and motivate measuring throughput and loss alongside RTT, rather than inferring useful rate from cwnd alone. They do not establish that retransmissions caused the entire goodput difference, and they do not establish optimal thresholds. The source files are results/20261018T035444Z_185.180.12.40_5201.csv and results/20261018T035817Z_185.180.12.40_5201.csv; calculations use run_statistics in comparison_plots.py. The 10% and 25% queue thresholds create a band in which the sender holds its window. Low delay allows probing; sustained queue growth causes a small reduction before loss. Loss triggers a larger reduction. The eight-RTT probe avoids permanently accepting an underestimated bandwidth estimate after a path gains capacity. The BDP gate limits routine growth when extra window stops increasing useful delivery. Startup provides initial discovery without an established bandwidth estimate.

Assignment 2 Q3(b): Algorithm design

The full baseline comparison reinforces the use of all three signals. In Dallas, Reno's median cwnd rose from 1,982 to 7,854 segments across repetitions while goodput stayed near 279-284 Mbit/s, motivating a throughput gate on routine window growth. On Netfiber, BBR's third run fell to 86.57 Mbit/s with 396 retransmissions despite mean RTT near the path baseline, motivating a separate loss response rather than relying on RTT alone. At 96.45.41.167, one BBR interval had zero acknowledged goodput and a 1,559.64 ms smoothed RTT, motivating a conservative response to sustained delay inflation as well. These are associations in sequential public-path tests, not proof of any mechanism or validation of the proposed thresholds. The analysis and source measurements are in results/q3-test/observations.md and its referenced CSV files.

The constants remain initial hypotheses. We retain the previously shared pseudocode and thresholds so this handoff stays consistent; the new data do not justify presenting those constants as tuned or optimal. Validate them in controlled tests and document any changes in Assignment 3. A rolling base RTT adapts after route changes but can mistake a persistent queue for propagation delay when old samples expire. A rolling bandwidth maximum also lags rapid capacity decreases.

Fairness and stability

Under equal RTTs, persistent backlog, similar packet sizes, and a shared drop-tail bottleneck, equal additive growth with proportional loss reductions can move flows toward similar windows. The queue response can reduce the amplitude of a congestion cycle. This is an expectation, not a proof of fairness or convergence.

Flows with different RTTs update at different frequencies, so RTT fairness is not guaranteed. A delay-sensitive flow may yield to CUBIC or Reno flows that maintain queues. Its estimated minimum RTT may include standing queues, and independent bandwidth estimates can overstate each flow's fair share. The throughput gate can also preserve an unfair initial allocation. Public Internet measurements cannot isolate these effects.

For a single flow on a fixed path, the hold band and rate smoothing should reduce rapid changes, but periodic probing and multiplicative reductions can produce a limit cycle rather than a fixed window. Delayed feedback, ACK compression, reverse-path congestion, random wireless losses, and changing routes can destabilize decisions. Evaluate equal and unequal RTT competing flows, utilization, queue delay, recovery, and Jain's fairness index before claiming stable or fair operation.

Assignment 2 Q3(b): Algorithm design

Assignment 3 handoff

Implement this state machine in a kernel congestion-control module or eBPF struct_ops program. A Python client cannot implement a kernel congestion window policy by selecting TCP_CONGESTION alone. Use kernel delivery/rate samples and ACK/recovery callbacks to supply the inputs above. Test that callbacks preserve the once-per-RTT and once-per-recovery rules. Record callback mappings, unavailable signals, parameter changes, exact kernel version, and the actual loaded algorithm name. The comparison command accepts extra algorithm names, so it can evaluate the custom algorithm alongside cubic, reno, and bbr with 10-second runs.